به نام خدا

ساختمان داده‌ها و الگوریتم‌ها (۴۰۲۵۴) ۲۲ آبان ۹۸

دانشگاه صنعتی شریف مدرس: مهدی صفرنژاد

آزمون میان‌ترم

زمان این امتحان ۱۵۰ دقیقه است. سؤالات امتحان در سه صفحه قرار دارند و مجموع نمرات سؤالات ۱۰۰ نمره است. توضیحات ابتدای سؤالات را به دقت بخوانید و پاسخ هر کدام از ۵ سؤال اصلی را در برگه‌های جداگانه بنویسید. پاسخ بخش‌های مختلف هر سؤال را نیز به ترتیب در برگهٔ پاسخ‌نامه بنویسید.

سؤال ۱. [۱۰ نمره] سؤال‌های جواب کوتاه.

آ. **[۵ نمره]** چرا الگوریتم مرتب‌سازی سریع در زبان برنامه‌نویسی سی‌پلاس‌پلاس و جاوا (برای مرتب کردن اعداد) به جای مرتب‌سازی ادغامی استفاده می‌شود. حداقل ۲ دلیل ذکر کنید.

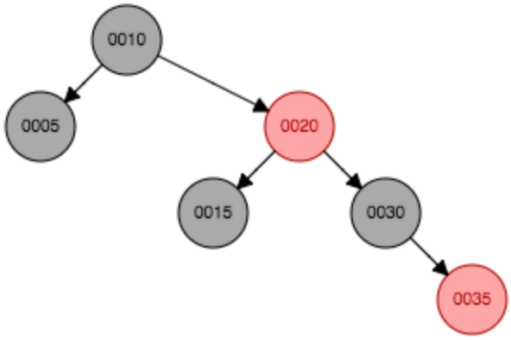
پاسخ:

۱. تقریبا ۲ برابر سریع‌تر است.

۲. حافظه اضافی زیادی نمی‌خواهد.

ب. **[۵ نمره]** یک درخت قرمز_سیاه مثال بزنید که AVL نباشد.

پاسخ: درختان زیادی وجود دارد. به عنوان مثال:



سؤال ۲. [۲۴ نمره] تحلیل مجانبی. در هر قسمت علاوه بر پاسخ درست، روابطی که استفاده کردید را نوشته و به طور مختصر توضیح دهید. در سؤال‌های زیر مبنای log برابر با ۲ می‌باشد.

آ. **[۶ نمره]** با استفاده از قضیهٔ اصلی، رابطهٔ زیر را حل کنید.

$$T(n) = ۴۹T(n/۷) + n^۲(\log n)^۳$$

پاسخ: $N^۲log^۴n$

ب. **[۶ نمره]** رابطهٔ بازگشتی زیر را حل کنید.

$$T(n) = T(\sqrt{n}) + \log n$$

پ. **[۶ نمره]** الگوریتمی داریم که زمان اجرای آن $O(n^{۲-(۳/۲)\phi} + n^{۲\beta} + n^{(۱-\beta)(۲-\phi)+۱/۵\beta})$ است. پارامترهای β و ϕ را می‌توانیم به دلخواه از اعداد بین صفر تا یک انتخاب کنیم. هدف این که با انتخاب پارامترهای مناسب زمان اجرای الگوریتم را تا حد ممکن کاهش دهیم. پارامترها مناسب و زمان اجرای الگوریتم با این پارامترها چقدر خواهد بود؟ (راهنمایی: زمانی الگوریتم کمترین زمان را دارد که توان سه جمله یکسان باشد.)

پاسخ: $\phi = \frac{۵}{۴} - \frac{\sqrt{\frac{۴۳}{۳}}}{۴} \quad \beta = \frac{۱}{۱۶} - \frac{\sqrt{۱۲۹}}{۱۶}$

طبق راهنمایی سوال کافیست دو معادله دو مجهول زیر را حل کنیم.

$$۲ - ۱/۵\phi = ۲\beta$$

$$۲\beta = (۱ - \beta)(۲ - \phi) + ۱/۵\beta$$

این معادلات دو مجموعه جواب دارند اما فقط یک مجموعه آن‌ها معتبر (در بازه ۰ و ۱) است.

نمره‌دهی: راه حل با جواب کاملا اشتباه (بدون شباهت به جواب اصلی) که فقط به معادلات درست اشاره کرده باشد ۱ نمره گرفته و سایر راه حل‌ها به تناسب نمره می‌گیرند.

ت. **[۶ نمره]** رابطهٔ بازگشتی زیر را حل کنید.

$$T(n) = T(n/\log n) + O(۱)$$

پاسخ: $O(\log n/\log\log n)$

می‌توان پاسخ بالا را به کمک درخت بازگشتی حدس زد. برای اثبات آن، تغییر متغیر

$$s = \log n \quad f(s) = f(s - \log s) + O(۱)$$

را در نظر بگیرید. می‌خواهیم نشان دهیم

$$f(s) = O(\frac{s}{\log s})$$

برای اثبات قضیه فوق ابتدا لم زیر را ثابت می‌کنیم

$$\exists A, x_۰ \forall x > x_۰ \quad A \log x \leq \log(x - \log x)$$

برای اثبات لم فوق دقت کنید که

$$\log(x - \log x) = \log(x) + \log(۱ - \frac{\log x}{x})$$

عبارت $log(۱ - \frac{\log x}{x})$ هنگامی که $x \to \infty$ به صفر میل می‌کند بنابراین به ازای ϵ به اندازه دلخواه کوچک نامنفی داریم

$$\exists x_۰ \forall x > x_۰ \quad \log(۱ - \frac{\log x}{x}) \geq -\epsilon \implies \log(x - \log x) \geq \log x - \epsilon \geq \log x(۱ - \frac{\epsilon}{\log x_۰})$$

پس کافیست داشته باشیم $A < ۱ - \frac{\epsilon}{\log x_۰}$. حال اگر دنباله t_n را به شکل $t_{n+۱} = t_n - \log t_n$ تعریف کنیم آنگاه $f(n) \in O(n^*)$ به گونه‌ای که $t_{n^*} = ۰$.

$$\log t_{n+۱} \geq A \log t_n \implies t_n^* \geq s - \log s \sum_{i=۱}^{n} A^i \implies \frac{۱ - A^{n^*}}{۱ - A} \leq \frac{s}{\log s}$$

تعریف می‌کنیم $A = ۱ - \epsilon$. آنگاه

$$۱ - A^{n^*} = ۱ - (۱ - \epsilon)^{n^*} \leq ۱ - (۱ - \frac{\epsilon n^*}{۲}) \implies \frac{\epsilon n^*}{۲} \leq \frac{\epsilon s}{\log s}$$

$$n^* \in O(\frac{s}{\log s}) \implies f(s) \in O(\frac{s}{\log s}) \implies T(n) \in O(\frac{\log n}{\log \log n})$$

نمره‌دهی: در این سوال به پاسخ‌هایی که با استفاده از درخت بازگشتی به جواب رسیده‌اند نیز اگر استدلال درست باشد نمره کامل و برای استدلال‌های نادرست نیز به تناسب راه حل نمره در نظر گرفته شده است.

سؤال ۳. **[۳۰ نمره] پیشنهاد الگوریتم یا داده‌ساختار مناسب.** در هر قسمت از این سؤال از شما خواسته می‌شود یک الگوریتم یا داده‌ساختار مناسب پیشنهاد دهید. این داده‌ساختار یا الگوریتم می‌تواند از مطالب گفته شده در درس، ترکیبی از آن‌ها و یا خلاقیتی باشد. علاوه بر کلیت داده‌ساختار و الگوریتم، زمان اجرا و دلیل انتخاب خود را نیز در ۳ الی ۵ خط به طور واضح توضیح دهید. در این سؤال نیاز به اثبات درستی الگوریتم و اثبات زمان اجرای الگوریتم نیست. همچنین در صورتی که زمان اجرای الگوریتم شما زمان سرشکن یا زمان تصادفی است، حتماً آن را بیان کنید.

آ. **[۵ نمره]** یک آرایه از اعداد داریم که از ابتدا تا جایی از آن صعودی و پس از آن نزولی است. چه الگوریتمی را برای مرتب کردن این آرایه مناسب است؟ زمان اجرای آن چقدر است؟

پاسخ: ابتدا با یک پیمایش عضو بیشینه‌ی آرایه را پیدا می‌کنیم، سپس اگر از اول آرایه تا این عضو را در یک آرایه در نظر بگیریم، و اعداد پس از این عضو را نیز به ترتیب برعکس در یک آرایه دیگر در نظر بگیریم، کافی است این ۲ قسمت را با هم ادغام کنیم(مانند مرتب‌سازی ادغامی) از آنجایی که ادغام از $O(n)$ است، زمان اجرای الگوریتم ما نیز از $O(n)$ خواهد بود.

نمره‌دهی: الگوریتم بالا نمره‌ی کامل را می‌گیرد و الگوریتم‌های دیگر با زمان اجرای بیش‌تر و راه‌حل‌های ناقص، متناسب با راه‌حل بخشی از نمره را گرفته‌اند.

ب. **[۵ نمره]** یک آرایه از اعداد داریم که اندیس هر عدد با اندیسی که پس از مرتب شدن کل آرایه پیدا می‌کند، حداکثر ۱۰ تا اختلاف دارد. بهترین الگوریتم برای مرتب‌سازی این آرایه از بین الگوریتم‌های مرتب‌سازی که در کلاس مطرح شد کدام است؟ زمان اجرای آن چقدر است؟

پاسخ: در صورتی که از مرتب سازی درجی استفاده کنیم، هر عضو زمانی که به آن می‌رسیم نیز حداکثر ۱۰ خانه با جایگاه اصلی خود فاصله دارد، پس در $O(1)$ به جایگاه خود می‌رسد، در نتیجه در مجموع زمان اجرا از $O(n)$ خواهد بود.

نمره‌دهی: اگر الگوریتم ارائه شده درست و از $O(n)$ باشد، نمره‌ی کامل را می‌گیرد، در غیر اینصورت متناسب با راه‌حل نمره می‌گیرد.

پ. **[۸ نمره]** یک آرایه به نام A و به طول n از اعداد داریم که لزوماً مرتب نیست. یک عدد x و یک مقدار صحیح و مثبت $k < n$ به ما داده شده است و می‌خواهیم k تا بزرگترین عدد موجود در A که کوچک‌تر از x هستند را در صورت وجود پیدا کنیم. توجه کنید x لزوماً در آرایه موجود نیست و ترتیب اعداد خروجی برای ما مهم نیست.

پاسخ: ابتدا با یک دور پیمایش آرایه اعدادی که بزرگ‌تر مساوی x هستند را حذف می‌کنیم سپس باید در آرایه باقی‌مانده k تا بزرگ‌ترین عدد را پیدا کنیم که این کار را هم در بهترین حالت می‌توان در زمان اجرای nlogk انجام داد. به این گونه که روی اعداد آرایه ثانویه پیمایش می‌کنیم و هر عدد را به یک هرم کمینه اضافه می‌کنیم و اگر اندازه درخت بیشتر از k شده بود عضو کمینه را از آن حذف می‌کنیم. به این ترتیب در هر مرحله k تا عضو بیشینه در هرم هستند و در نهایت می‌توانیم این اعداد را خروجی دهیم.

ت. **[۶ نمره]** در این مسئله با یک داده ساختار جدید سر و کار داریم که مشابه هرم است ولی خاصیت «تقریباً کامل» بودن هرم را ندارد. در عوض مشابه درخت AVL، می‌دانیم اختلاف ارتفاع زیردرخت فرزندان هر رأس حداکثر یک واحد با هم اختلاف دارد (مشابه درخت AVL ارتفاع رأس‌هایی که وجود ندارند را ۱− درنظر می‌گیریم). الگوریتمی برای درج یک عنصر جدید در این داده ساختار پیشنهاد دهید.

پاسخ: ابتدا مقدار جدید را به درخت اضافه می‌کنیم. سپس با استفاده از up buble تا جای ممکن راس جدید را بالا می‌بریم. اگر در نهایت در مسیر طی شده همه راس‌ها ویژگی درخت AVL را داشتند یعنی درخت جدید هم ویژگی گفته شده را دارد. در غیر این صورت پایین‌ترین راسی که این ویژگی را دارد را در نظر می‌گیریم و مشابه rotate در درخت AVL ارتفاع درخت را یکی کم می‌کنیم.

نمره‌دهی:

ث. **[۶ نمره]** n شیء داریم که شیء iاُم دارای کلید (a_i, b_i) است که a_iها و b_i اعداد صحیح و نامنفی کمتر از n هستند. می‌خواهیم این اشیاء را مرتب کنیم. وقتی می‌خواهیم دو شیء را با هم مقایسه کنیم از تابع $f(a,b) = (۱ + \frac{a}{n}) \cdot ۲^b$ روی کلید هر کدام استفاده می‌کنیم و شیءای کوچکتر است که خروجی f روی کلید آن کوچکتر باشد. الگوریتمی برای مرتب‌سازی این اشیاء پیشنهاد دهید.

پاسخ: ابتدا ثابت می‌کنیم که اگر دو تا b نابرابر باشند مقدار f در آنی که b اش بزرگتر است بیشتر می‌باشد.

$$b_۱ < b_۲, a_۱ < n, a_۲ < n$$

$$f(a_۱, b_۱) = (۱ + \frac{a_۱}{n})۲^{b_۱} < ۲ \times ۲^{b_۱} \leq ۲^{b_۲} \leq (۱ + \frac{a_۲}{n})۲^{b_۲} = f(a_۱, b_۱)$$

همچنین برای b های برابر باید طبق a مقایسه شوند.

برای حل می‌توانیم آن‌را به صورت عدد دو رقمی ببنیم در مبنای n چرا که $a, b < n$ و از روش مرتب‌سازی مبنایی استفاده کنیم که $O(n)$ به ما جوابی می‌دهد.

نمره‌دهی: اگر به موضوع اولویت مقایسه b اشاره شده‌باشد ۳ نمره تعلق می‌گیرد. اگر به مرتب‌سازی مبنایی اشاره شده باشد هم ۳ نمره به شما تعلق می‌گیرد.

راه‌حل‌هایی که الگوریتم با مرتبه زمانی بدتر از $O(n)$ پیشنهاد داده‌اند صفر در نظر گرفته‌می‌شوند.
همچنین اشاره به اثبات اولویت مقایسه b بخشی از نمره را دارد.

سؤال ۴. **[۲۴ نمره]** آرایه‌ای از اعداد داریم و می‌خواهیم آن را مرتب کنیم. در هر مرحله می‌توانیم یک زیرمجموعه از اندیس‌های آرایه انتخاب کنیم و آن عناصر را با حفظ ترتیب به ابتدای آرایه منتقل کنیم. به طور مثال اگر آرایه $\langle ۵۴, ۱۳۲, ۴۷, ۹۶ \rangle$ باشد و اندیس‌های ۱ و ۳ را انتخاب کنیم، عناصر ۵۴ (که اندیس ۱ دارد) و ۴۷ که اندیس ۳ دارد با حفظ ترتیب به ابتدای آرایه می‌آیند و نتیجه می‌شود $\langle ۵۴, ۴۷, ۱۳۲, ۹۶ \rangle$. در پاسخ به موارد زیر اثبات دقیق ادعای خود را بیان کنید.

آ. **[۱۲ نمره]** ثابت کنید برای مرتب‌سازی این آرایه در بدترین حالت، حداقل $\Omega(\log n)$ مرحله نیاز است.

پاسخ: P_n را آرایه‌ای نزولی از n عضو در نظر میگیریم.

لم: برای مرتب کردن P_n (به صورت صعودی) حداقل به $\log n$ مرحله نیاز است.

اثبات. لم را به استقرا روی n ثابت می کنم.

پایه: برای $n = ۱$ صفر مرحله نیاز است.

گام: فرض میکنم حکم به ازای $n < k$ برقرار است.

P_k را در نظر بگیرید بعد از یک مرحله، یا حداقل $\frac{k}{۲}$ تا هستند که انتخاب نشده اند یا حداقل $\frac{k}{۲}$ تا هستند که انتخاب شده اند این $\frac{k}{۲}$ تا در هر دو صورت ترتیبشان نسبت به هم عوض نشده اگر در ادامه بقیه اعضا را درنظر نگیریم برای مرتب کردن این $\frac{k}{۲}$ تا عضو طبق فرض استقرا به حداقل $\log \frac{k}{۲} = \log k - ۱$ حرکت نیاز داریم که اگر حرکت اول را نیز حساب کنیم پس برای مرتب کردن $\frac{k}{۲}$ تا حداقل به $\log k$ تا عملیات نیاز داریم.

نمره‌دهی: آوردن مثالی که در آن حداقل به $\Omega(\log n)$ مرحله نیاز است ۴ نمره دارد (در بعضی راه‌حل‌ها برای این قسمت اثبات وجودی آوردند، آن‌ها هم نمره را گرفته‌اند).
۸ نمره باقیمانده به اثبات تعلق گرفته.

ب. **[۱۲ نمره]** الگوریتمی ارائه دهید که هر آرایه را در حداکثر $O(\log n)$ مرحله مرتب کند. درستی الگوریتم خود را اثبات کنید.

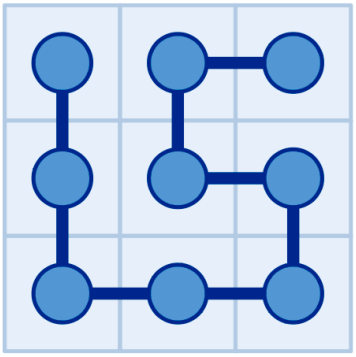
پاسخ: میانه اعداد را در نظر بگیرید و آن را x بنامید، اعدادمان را به دو بخش کوچکتر مساوی x و بیشتر از x تقسیم می‌کنیم. سپس هر یک از این دو بخش را به صورت **همزمان** حل می‌کنیم، یعنی نهایتا

در آرایه‌مان، اعداد کوچکتر از x، نسبت به یکدیگر مرتب شده‌اند و اعداد بزرگتر از x نیز نسبت به یکدیگر مرتب شده‌اند. در گام آخر نهایتا اندیس‌های کوچکتر از x را انتخاب کرده و آن‌ها را به ابتدای آرایه می‌آوریم.

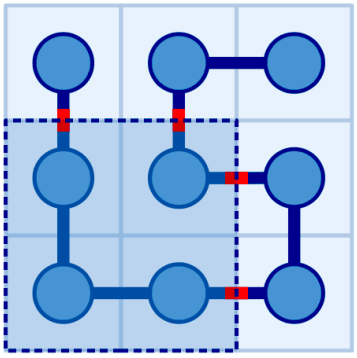
$$T(N) = T(\left\lceil \frac{N}{۲} \right\rceil) + ۱ \rightarrow T(N) \in \theta(\log N)$$

نمره‌دهی: روش دیگر حل به کمک نمایش مبنای دو و استفاده از مرتب‌سازی مبنایی بود که آن نیز در صورت درستی، نمره کامل را کسب کرده است. راه‌حل‌های غلط نیز نمره‌ای تقریبا برابر با صفر گرفته‌اند.

سؤال ۵. [۱۲ نمره] در بازی اسنِیک، یک جدول $n \times n$ داریم که یک مار تمام خانه‌های آن را مشابه شکل زیر پوشانده است و از یک خانه نیز دوبار عبور نکرده است.



شما در هر مرحله می‌توانید به طور دلخواه یک زیرجدول را انتخاب کنید و این سؤال را بپرسید که زوجیت تعداد بارهایی که مار مرز این جدول را قطع می‌کند چند بار است؟ به طور مثال در شکل زیر ما چهار بار مرز زیرجدول را رد می‌کند و جواب سؤال زوج است.



الگوریتمی طراحی کنید که با $O(n)$ پرسش سر و ته مار را پیدا کند.

پاسخ: ابتدا توجه کنید که جواب یک زیرجدول فرد است، اگر و تنها اگر دقیقا یکی از سر یا ته مار درون آن باشد؛ و زوج است، اگر و فقط اگر هر دو یا هیچکدام درون زیر جدول باشند.

اکنون، ابتدا هر سطر و هر ستون را به عنوان زیرجدول می‌پرسیم. اگر سر و ته مار در دو سطر مختلف باشند، آن دو سطر جواب فرد و بقیه جواب زوج می‌دهند و اگر در یک سطر باشند، همه سطرها جواب زوج می‌دهند. به طور مشابه برای ستون ها نیز همین اتفاق می‌افتد. چون سر و ته مار در دو خانه مختلف‌اند، میان سطرها یا ستون‌ها (یا هر دو) دو جواب فرد پیدا می‌شود. یعنی سر و ته مار در آن دو سطر (یا دو ستون) هستند. حال کافیست تمام خانه‌های آن دو سطر (یا دو ستون) را تک تک بپرسیم تا سر و ته مار را پیدا کنیم. زمان اجرای این الگوریتم $۴ \times O(n) = O(n)$ است.

نمره‌دهی: برای بیان پاراگراف اول ۳ نمره در نظر گرفته شده و اثبات دقیق آن الزامی نیست. اگر الگوریتم پیشنهاد شده درست باشد، نمره این بخش هم داده می‌شود. در مجموع، سختگیری زیادی نشده و به تمام الگوریتم‌های درست، نمره کامل داده شده است و از راه‌حل‌های ناقص متناسب با مقدار ناقص بودن آن‌ها نمره کم شده است.

موفق باشید